AgentFlow User Guide

A comprehensive guide to building AI agent systems with AgentFlow — the most feature-complete AI agent framework for Go.

Table of Contents

1. Overview
2. Installation
3. Project Structure
4. Quick Start
5. Core Concepts
6. Creating Agents
7. Defining Tools
8. Building Multi-Agent Systems
9. Orchestration Strategies
10. Hierarchical Teams
11. LLM Providers
12. RAG (Retrieval-Augmented Generation)
13. Memory
14. Guardrails
15. Human-in-the-Loop
16. Tracing
17. Sessions (Persistence)
18. Eval / Testing Harness
19. MCP Integration
20. Conversation & Context
21. Error Handling & Reliability
22. Logging
23. Testing with Mocks
24. Examples Reference
25. API Reference

26. FAQ
27. Comparison

Overview

AgentFlow is a production-grade AI agent framework written in Go. It lets you build autonomous AI agents that collaborate to solve complex tasks — with true concurrency, zero external dependencies, and clean abstractions.

Key Design Principles

Principle	How
Zero dependencies	Pure Go standard library — no external packages
No hanging	Every operation has a `context.Context` deadline
True concurrency	Goroutines for parallel agents
Clean abstractions	Agent, Tool, Manager, Conversation
LLM-agnostic	Pluggable LLM interface — OpenAI, Mistral, Groq, Ollama, or custom
MCP-native	First-class Model Context Protocol support
RAG built-in	Load docs from files, URLs, strings — no vector DB needed
Testable	MockLLM + MockMCPTransport for deterministic tests
Full observability	Memory, tracing, guardrails, HITL, sessions, eval

What You Can Build

- Strategic analysis pipelines (research → synthesize → recommend)
- RAG-powered documentation assistants
- Data processing workflows with external APIs
- Customer support with specialized agents and guardrails
- Content generation pipelines with human oversight
- Any task that benefits from multiple specialized AI agents

Feature Summary

Category	Features
Core	Agents, Tools, Managers, Conversations, per-agent LLM
Orchestration	Sequential, Parallel, RoundRobin, LLM Router
RAG	Files, URLs, strings, directories, dynamic uploads, keyword + vector search
Memory	Multi-layer (fact, episode, instruction, summary), TTL, tags, search
Guardrails	10 built-in (PII filter, injection blocker, RedactPII, JSON validation, etc.)
HITL	Approve/Reject/Edit/Abort, 4 policies, CLI + Channel handlers
Tracing	OTel-style hierarchical spans, 8 span kinds, stats, JSON export
Sessions	Save/resume, checkpoints, file + memory backends
Eval	12 assertions, regression tracking, tag-based filtering, report comparison
MCP	Auto-discovery, multi-server, manager propagation, mock transport

Installation

```
# Create a new Go project
mkdir my-agents && cd my-agents
go mod init my-agents

# Add AgentFlow
go get github.com/neranjsubramanian/agentflow
```

Requirements

- Go 1.24 or later
- An LLM API key (Mistral, OpenAI, Groq, etc.) — or use MockLLM for testing

Project Structure

AgentFlow separates the **framework** (the library you import) from the **examples** (programs that show how to use it).

```
agentflow/
├── *.go                ← The framework. You import this.
├── *_test.go           ← 156 unit/integration tests
├── go.mod              ← Module: github.com/neranjsubramanian/agentflow
├── LICENSE
├── README.md
│
├── docs/               ← Documentation
│   ├── user-guide.md
│   └── generate_pdf.py
│
└── examples/           ← Runnable programs that use the framework
    ├── basic/          ← Simplest: one agent, one question
    ├── single_agent/   ← Agent with custom tools
    ├── two_agent/      ← Sequential pipeline (researcher → writer)
    ├── parallel_agents/ ← 6 agents running in parallel
    ├── llm_router/     ← LLM dynamically picks which agent runs
    ├── external_api/   ← Agent calling HTTP APIs as tools
    ├── mcp_agent/      ← MCP protocol for dynamic tool discovery
    ├── rag/            ← RAG: files, strings, URLs, dynamic uploads
    ├── showcase/       ← Multi-agent pipeline with web search
    ├── frameworks/     ← 7 strategy agents + synthesizer
    └── webui/          ← WhatsApp-style chat UI with WebSocket
```

To use the framework in your own project:

```
import "github.com/neranjsubramanian/agentflow"
```

To run an example:

```
cd examples/basic
go run main.go --demo
```

Quick Start

Step 1: Your First Agent (No API Key Needed)

```
package main

import (
    "context"
    "fmt"

    "github.com/neranjsubramanian/agentflow"
)

func main() {
    mock := &agentflow.MockLLM{}
    mock.PushResponse(agentflow.LLMResponse{
        Content: "Go is great for concurrent backend services.",
    })

    agent := agentflow.NewAgent(agentflow.AgentConfig{
        ID:   "assistant",
        Role: "Helpful Assistant",
        Goal: "Answer questions clearly and concisely.",
        LLM:  mock,
    })

    result, err := agent.Run(context.Background(), "Why should I use Go?", nil)
    if err != nil {
        panic(err)
    }
    fmt.Println(result.Content)
}
```

Step 2: With a Real LLM

```
llm := agentflow.NewMistral(agentflow.OpenAIConfig{
    Model: "mistral-small-latest",
})

agent := agentflow.NewAgent(agentflow.AgentConfig{
    ID:   "researcher",
    Role: "Research Assistant",
    Goal: "Help users understand technical topics.",
    LLM:  llm,
})

result, _ := agent.Run(ctx, "Explain microservices vs monoliths", nil)
fmt.Println(result.Content)
```

Set your key first: `export MISTRAL_API_KEY=your-key-here`

Step 3: Multi-Agent Pipeline

```
researcher := agentflow.NewAgent(agentflow.AgentConfig{
    ID: "researcher", Role: "Researcher", Goal: "Find data", LLM: llm,
})
writer := agentflow.NewAgent(agentflow.AgentConfig{
    ID: "writer", Role: "Writer", Goal: "Write clear reports", LLM: llm,
})

pipeline := agentflow.NewManager(agentflow.ManagerConfig{
    ID:       "pipeline",
    Strategy: agentflow.StrategySequential,
    Agents:   []agentflow.Runnable{researcher, writer},
})

result, _ := pipeline.Run(ctx, "Write a report on the AI market", nil)
```

Core Concepts

AgentFlow has five core building blocks:

```
┌─────────┐      ┌──────┐      ┌─────────┐
│  Agent  │──────►│ Tool │      │   MCP   │
└────┬────┘      └──────┘      └────┬────┘
     │                              │
     │  tools auto-discovered       │
     ◄──────────────────────────────┘
     │
┌────▼────┐      ┌──────────────┐
│ Manager │──────►│ Conversation │
└─────────┘      └──────────────┘
```

Concept	What It Is
Agent	An autonomous unit with a role, goal, tools, and an LLM
Tool	A function an agent can call (web search, API, database)
Manager	Orchestrates a group of agents with a strategy
Conversation	Thread-safe message history shared between agents
MCP	External tool server with auto-discovered tools via protocol

The Runnable Interface

Both Agent and Manager implement `Runnable`:

```
type Runnable interface {
    GetID() string
    Run(ctx context.Context, input string, conv *Conversation) (*Result, error)
}
```

This enables **hierarchical composition**: anywhere you use an Agent, you can substitute a Manager (team of agents).

How an Agent Runs (ReAct Pattern)

AgentFlow agents follow the **ReAct** (Reason → Act → Observe) pattern:

1. Builds a **system prompt** from Role, Goal, and Backstory
2. Injects **memory** context (if configured)
3. Retrieves **RAG** chunks (if configured)
4. Runs **input guardrails** (if configured)
5. Adds **conversation context** and the current input
6. Calls the **LLM**
7. If LLM requests **tool calls**:
8. Checks **HITL** approval (if configured)
9. Executes tools
10. Feeds results back → calls LLM again (loop)
11. When LLM returns a **final message** (no tool calls):
12. Runs **output guardrails** (if configured)
13. Stores result in **memory** (if configured)
14. Records output in the **Conversation**

15. Returns `*Result`

All steps are **traced** if a Tracer is present in the context.

Creating Agents

AgentConfig — Full Reference

```
agent := agentflow.NewAgent(agentflow.AgentConfig{
    // Required
    ID:   "unique-id",
    Role: "Market Analyst",
    Goal: "Analyze market trends with data",
    LLM:  llm,

    // Optional — behavior
    Backstory:    "15 years at McKinsey analyzing tech markets.",
    Tools:        []agentflow.Tool{searchTool, calcTool},
    MaxToolCalls: 10,               // safety limit (default: 10)
    Timeout:      2 * time.Minute,  // hard deadline (default: 2m)
    Logger:       slog.Default(),

    // Optional — MCP
    MCPServers: []agentflow.MCPServerConfig{
        {Name: "search", Command: "./search_server"},
    },
    DisableMCP: false,

    // Optional — advanced features
    Memory:          memoryStore,     // multi-layer memory
    RAG:             ragConfig,       // retrieval-augmented generation
    InputGuardrails:  inputChain,    // validate before LLM
    OutputGuardrails: outputChain,   // validate before return
    HITL:            &hitlConfig,     // human approval for tools
})
```

Crafting Good Agents

Role — Tell the agent *what* it is:

```
"Senior Market Research Analyst"
"Competitive Intelligence Specialist"
"Full-Stack Software Engineer"
```

Goal — Tell the agent *what to achieve* (be specific):

```
"Research market size, growth rate, and key trends for the given industry.
 Provide specific numbers. Use web_search for current data."
```

Backstory — Give it *context* that shapes behavior:

```
"You have 15 years of experience at McKinsey analyzing tech markets.
 You always quantify your findings and cite sources."
```

Defining Tools

Tools give agents capabilities beyond text generation. When the LLM decides it needs data, it calls the tool through function calling.

Basic Tool

```
weatherTool := agentflow.Tool{
    Name:        "get_weather",
    Description: "Get the current weather for a city",
    Parameters: map[string]agentflow.ToolParam{
        "city": {Type: "string", Description: "City name", Required: true},
    },
    Execute: func(args map[string]any) (string, error) {
        city := args["city"].(string)
        return fmt.Sprintf("Weather in %s: 22°C, sunny", city), nil
    },
}
```

HTTP API Tool

```
marketAPI := agentflow.Tool{
    Name:        "get_market_data",
    Description: "Fetch market size and growth data for a specific industry",
    Parameters: map[string]agentflow.ToolParam{
        "industry": {Type: "string", Description: "Industry name", Required: true},
        "region":   {Type: "string", Description: "Geographic region"},
    },
    Execute: func(args map[string]any) (string, error) {
        industry := args["industry"].(string)
        resp, err := http.Get(fmt.Sprintf("https://api.example.com/market?q=%s", industry))
        if err != nil {
            return "", err
        }
        defer resp.Body.Close()
        body, _ := io.ReadAll(resp.Body)
        return string(body), nil
    },
}
```

Tool Design Guidelines

1. **Name**: Short, snake_case (`web_search` , `get_market_data`)
2. **Description**: Tell the LLM *when* to use it — this is the most important field
3. **Parameters**: Include descriptions — the LLM uses them to fill in arguments
4. **Execute**: Return errors instead of panicking; return structured text the LLM can reason about

---

Building Multi-Agent Systems

ManagerConfig Reference

```
mgr := agentflow.NewManager(agentflow.ManagerConfig{
    // Required
    ID:       "team-id",
    Strategy: agentflow.StrategyParallel,
    Agents:   []agentflow.Runnable{agent1, agent2},

    // Optional
    MaxTurns:   20,               // for RoundRobin/LLMRouter (default: 20)
    Timeout:    10 * time.Minute, // hard deadline (default: 10m)
    LLM:        llm,              // required only for LLMRouter
    Logger:     slog.Default(),
    MCPServers: []agentflow.MCPServerConfig{...},
})
```

Dynamic Agent Addition

```
mgr := agentflow.NewManager(agentflow.ManagerConfig{
    ID: "team", Strategy: agentflow.StrategySequential,
    Agents: []agentflow.Runnable{agent1},
})
mgr.AddAgent(agent2)
mgr.AddAgent(agent3)
```

Orchestration Strategies

Sequential

Agents run one after another. Each gets the previous agent's output as input.

```
mgr := agentflow.NewManager(agentflow.ManagerConfig{
    Strategy: agentflow.StrategySequential,
    Agents:   []agentflow.Runnable{researcher, writer, editor},
})
```

Best for: Pipelines, draft → edit → publish, research → synthesize.

Parallel

All agents run **concurrently** via goroutines. Results are merged. Resilient — if one agent fails, others continue.

```
mgr := agentflow.NewManager(agentflow.ManagerConfig{
    Strategy: agentflow.StrategyParallel,
    Agents:   []agentflow.Runnable{porter, swot, pestel, vrio, bmc, blueOcean},
})
```

Best for: Independent analyses, gathering data from multiple sources.

RoundRobin

Cycles through agents until MaxTurns or a `[DONE]` / `[FINISH]` signal.

```
mgr := agentflow.NewManager(agentflow.ManagerConfig{
    Strategy: agentflow.StrategyRoundRobin,
    Agents:   []agentflow.Runnable{proposer, critic, refiner},
    MaxTurns: 10,
})
```

Best for: Iterative refinement, debate, review cycles.

LLMRouter

An LLM dynamically decides which agent to call next or when to finish.

```
mgr := agentflow.NewManager(agentflow.ManagerConfig{
    Strategy: agentflow.StrategyLLMRouter,
    Agents:   []agentflow.Runnable{researcher, analyst, writer},
    LLM:      llm,
    MaxTurns: 15,
})
```

Best for: Dynamic workflows where the next step depends on results.

Hierarchical Teams

Because Manager implements `Runnable`, you can nest managers:

```
researchTeam := agentflow.NewManager(agentflow.ManagerConfig{
    ID:       "research-team",
    Strategy: agentflow.StrategyParallel,
    Agents:   []agentflow.Runnable{marketAgent, techAgent, competitorAgent},
})

fullPipeline := agentflow.NewManager(agentflow.ManagerConfig{
    ID:       "pipeline",
    Strategy: agentflow.StrategySequential,
    Agents:   []agentflow.Runnable{researchTeam, synthesizerAgent},
})
```

This creates:

```
pipeline (Sequential)
├── research-team (Parallel)
│   ├── market-agent
│   ├── tech-agent
│   └── competitor-agent
└── synthesizer-agent
```

LLM Providers

AgentFlow works with any OpenAI-compatible API. Each agent can use a different provider.

Built-in Providers

```
// OpenAI
llm := agentflow.NewOpenAI(agentflow.OpenAIConfig{
    Model: "gpt-4o-mini",  // reads OPENAI_API_KEY from env
})

// Mistral
llm := agentflow.NewMistral(agentflow.OpenAIConfig{
    Model: "mistral-small-latest",  // reads MISTRAL_API_KEY from env
})
```

Any OpenAI-Compatible API

```
// Groq
llm := agentflow.NewOpenAI(agentflow.OpenAIConfig{
    BaseURL: "https://api.groq.com/openai/v1",
    APIKey:  os.Getenv("GROQ_API_KEY"),
    Model:   "llama-3.1-70b-versatile",
})

// Ollama (local, no API key needed)
llm := agentflow.NewOpenAI(agentflow.OpenAIConfig{
    BaseURL: "http://localhost:11434/v1",
    Model:   "llama3.1",
    APIKey:  "ollama",
})
```

Mock LLM (Testing)

```
mock := &agentflow.MockLLM{}
mock.PushResponse(agentflow.LLMResponse{Content: "Hello!"})
mock.PushResponse(agentflow.LLMResponse{
    ToolCalls: []agentflow.ToolCall{
        {ID: "tc1", Function: "search", Args: map[string]any{"q": "test"}},
    },
})
mock.PushResponse(agentflow.LLMResponse{Content: "Final answer"})
```

Per-Agent Providers

```
analyst  := agentflow.NewAgent(agentflow.AgentConfig{LLM: mistralLLM, ...})
writer   := agentflow.NewAgent(agentflow.AgentConfig{LLM: openaiLLM, ...})
reviewer := agentflow.NewAgent(agentflow.AgentConfig{LLM: ollamaLLM, ...})
```

Custom LLM Provider

Implement the `LLM` interface to use any model:

```
type LLM interface {
    Chat(ctx context.Context, messages []Message, tools []Tool) (*LLMResponse, error)
}
```

RAG

RAG (Retrieval-Augmented Generation) lets agents answer questions grounded in your documents. AgentFlow has a full RAG pipeline built in — no external vector database required.

How It Works

1. **Ingest**: Load documents → split into chunks → store in vector store
2. **Query**: When agent receives a question, relevant chunks are retrieved
3. **Generate**: LLM answers using retrieved context as grounding

Loading Documents

From Files

```
store := agentflow.NewInMemoryVectorStore()

loader := agentflow.NewTextLoader("docs/product-spec.md")
pipeline := agentflow.NewIngestionPipeline(agentflow.IngestionConfig{
    Loader:   loader,
    Chunker:  agentflow.DefaultChunker(),
    Embedder: agentflow.NewNoOpEmbedder(),
    Store:    store,
})
stats, _ := pipeline.Ingest(ctx)
fmt.Printf("Loaded %d docs → %d chunks\n", stats.Documents, stats.Chunks)
```

From a Directory

```
loader := agentflow.NewDirectoryLoader("docs/", []string{".md", ".txt"})
```

From a URL

```
urlLoader := agentflow.NewURLLoader("https://example.com/docs/api-reference")
docs, _ := urlLoader.Load(ctx)
agentflow.IngestDocuments(ctx, store, agentflow.DefaultChunker(), agentflow.NewNoOpEmbedder
```

From Multiple URLs

```
loader := agentflow.NewMultiURLLoader(
    "https://example.com/docs/page1",
    "https://example.com/docs/page2",
    "https://example.com/blog/article",
)
```

From Strings (In-Memory)

```
loader := agentflow.NewStringsLoader(map[string]string{
    "company-policy.md": "All employees must...",
    "product-faq.md":    "Q: How does billing work?...",
})
```

Creating the RAG Agent

```
agent := agentflow.NewAgent(agentflow.AgentConfig{
    ID:   "doc-assistant",
    Role: "Documentation Assistant",
    Goal: "Answer questions using the knowledge base. Always cite sources.",
    LLM:  llm,
    RAG:  agentflow.SimpleRAG(store, 5),  // retrieve top 5 chunks per query
})

result, _ := agent.Run(ctx, "How do I deploy the app?", conv)
```

`SimpleRAG` creates a keyword-based retriever — no embeddings API needed. For semantic search, use an embedder (see below).

Adding Documents Mid-Conversation

Documents can be added at any time — no restart required:

```
newDoc := agentflow.Document{
    Content: "Version 3.0 adds streaming support...",
    Source:  "release-notes-v3.md",
}
agentflow.IngestDocuments(ctx, store, agentflow.DefaultChunker(), agentflow.NewNoOpEmbedder

// The next query will search the new document too
result, _ := agent.Run(ctx, "What's new in v3?", conv)
```

RAG from a URL Mid-Conversation

```
urlLoader := agentflow.NewURLLoader("https://example.com/blog/new-feature")
docs, _ := urlLoader.Load(ctx)
agentflow.IngestDocuments(ctx, store, agentflow.DefaultChunker(), agentflow.NewNoOpEmbedder
```

Chunkers

Chunker	Use Case
`DefaultChunker()`	Sensible default (recursive, 500 chars, 50 overlap)
`NewFixedSizeChunker(size, overlap)`	Fixed character windows
`NewParagraphChunker()`	Split on paragraph boundaries
`NewMarkdownChunker()`	Split on Markdown headings
`NewRecursiveChunker(size, overlap)`	Try headings, then paragraphs, then sentences

Embedders

The default `NoOpEmbedder` uses keyword matching (no API needed). For semantic similarity, use an embedder:

```
// Mistral embeddings
embedder := agentflow.NewMistralEmbedder(agentflow.EmbedderConfig{
    Model: "mistral-embed",
})

// OpenAI embeddings
embedder := agentflow.NewOpenAIEmbedder(agentflow.EmbedderConfig{
    Model: "text-embedding-3-small",
})
```

Retrievers

Retriever	What It Does
`NewKeywordRetriever(store)`	TF-IDF-style keyword matching
`NewVectorRetriever(store)`	Cosine similarity on embeddings
`NewHybridRetriever(store, alpha)`	Blend keyword + vector (alpha=0.7 means 70% vector)

Advanced RAG Configuration

```
agent := agentflow.NewAgent(agentflow.AgentConfig{
    RAG: &agentflow.RAGConfig{
        Retriever:   agentflow.NewHybridRetriever(store, 0.7),
        TopK:        10,
        MinScore:    0.3,
        CiteSources: true,
        QueryRewriter: func(input string) string {
            return "technical documentation: " + input
        },
    },
})
```

Memory

Multi-layer memory gives agents context from past interactions. Memories are automatically recalled before each LLM call based on relevance.

Setup

```
memory := agentflow.NewInMemoryStore()

memory.Store(ctx, agentflow.MemoryEntry{
    Kind:    agentflow.MemoryFact,
    Content: "The user prefers Go over Python",
    Tags:    []string{"preference"},
})
memory.Store(ctx, agentflow.MemoryEntry{
    Kind:    agentflow.MemoryInstruction,
    Content: "Always format code examples with syntax highlighting",
})

agent := agentflow.NewAgent(agentflow.AgentConfig{
    ID:     "assistant",
    Role:   "Personal Assistant",
    Goal:   "Help the user with context from past interactions",
    LLM:    llm,
    Memory: memory,
})
```

Memory Kinds

Kind	Purpose	Example
`MemoryFact`	Things the agent should know	"User is on the Pro plan"
`MemoryEpisode`	Past events	"Analyzed competitor X on Jan 5"
`MemoryInstruction`	Standing orders	"Always respond in bullet points"
`MemorySummary`	Compressed histories	"Past 10 conversations focused on..."

Features

- **Keyword search** with relevance scoring and priority boosting
- **Tag filtering** — retrieve memories by tags
- **TTL expiration** — memories can auto-expire
- **Session/agent scoping** — isolate memories per session or agent
- **Export/Import** — serialize to JSON for backup or transfer

Searching Memory

```
results, _ := memory.Search(ctx, "deployment process", 5)
for _, m := range results {
    fmt.Printf("[%s] %s\n", m.Kind, m.Content)
}
```

TTL and Scoping

```
memory.Store(ctx, agentflow.MemoryEntry{
    Kind:      agentflow.MemoryFact,
    Content:   "Temporary discount code: SAVE20",
    TTL:       24 * time.Hour,
    SessionID: "session-123",
    AgentID:   "sales-agent",
})
```

Guardrails

Validate and transform inputs before the LLM sees them, and outputs before they reach the user.

Setup

```
agent := agentflow.NewAgent(agentflow.AgentConfig{
    ID: "safe-agent", Role: "Assistant", Goal: "Help safely", LLM: llm,

    InputGuardrails: agentflow.NewGuardrailChain(
        agentflow.ContentFilter("injection", "pii"),
        agentflow.MaxLength(10000),
    ),

    OutputGuardrails: agentflow.NewGuardrailChain(
        agentflow.RedactPII(),
        agentflow.MinLength(20),
        agentflow.BlockKeywords("confidential", "internal only"),
    ),
})
```

Built-in Guardrails

Guardrail	What It Does
`MaxLength(n)`	Reject if input/output exceeds n characters
`MinLength(n)`	Reject if too short
`BlockPatterns(regex...)`	Block matching regex patterns
`RequirePatterns(regex...)`	Require matching patterns
`BlockKeywords(words...)`	Block specific words
`RequireJSON(fields...)`	Require valid JSON with specific fields
`ContentFilter(types...)`	Block PII, prompt injection
`RedactPII()`	Replace emails, phones, SSNs with `[REDACTED]`
`TrimWhitespace()`	Clean up whitespace
`CustomGuardrail(fn)`	Your own validation function

Custom Guardrail

```
profanityFilter := agentflow.CustomGuardrail("profanity-filter",
    func(ctx context.Context, input string) (string, error) {
        for _, word := range badWords {
            if strings.Contains(strings.ToLower(input), word) {
                return "", fmt.Errorf("blocked: inappropriate language")
            }
        }
        return input, nil
    },
)

agent := agentflow.NewAgent(agentflow.AgentConfig{
    OutputGuardrails: agentflow.NewGuardrailChain(profanityFilter),
})
```

Human-in-the-Loop

Pause agent execution for human approval before tool calls.

Setup

```
agent := agentflow.NewAgent(agentflow.AgentConfig{
    ID: "careful-agent", Role: "Analyst", Goal: "Analyze with oversight",
    LLM: llm, Tools: tools,

    HITL: &agentflow.HITLConfig{
        Policy:         agentflow.HITLPolicyDangerous,
        DangerousTools: []string{"delete_file", "execute_command"},
        Handler:        agentflow.NewCLIApprovalHandler(),
    },
})
```

Decisions

Decision	Effect
Approve	Execute the tool call as-is
Reject	Skip this tool call
Edit	Modify the tool arguments before executing
ApproveAll	Auto-approve all future tool calls in this run
Abort	Stop the entire agent run

Policies

Policy	When It Asks
`HITLPolicyNone`	Never (default)
`HITLPolicyAllTools`	Before every tool call
`HITLPolicyDangerous`	Only for tools in the dangerous list
`HITLPolicyCustom`	Your function decides

Channel Handler (for WebSocket/UI)

```
handler := agentflow.NewChannelHITLHandler()

// In your WebSocket handler:
go func() {
    req := <-handler.Requests()
    fmt.Printf("Agent wants to call %s with %v\n", req.ToolName, req.Args)
    handler.Respond(agentflow.HITLResponse{Decision: agentflow.HITLApprove})
}()
```

Audit Trail

```
hitl := agentflow.NewHITLController(config)
// ... after agent runs ...
for _, entry := range hitl.History() {
    fmt.Printf("%s: %s → %s\n", entry.ToolName, entry.Decision, entry.Reason)
}
```

Tracing

OpenTelemetry-style hierarchical tracing for every agent, tool, LLM call, RAG retrieval, and more.

Setup

```
tracer := agentflow.NewTracer()
ctx := agentflow.ContextWithTracer(context.Background(), tracer)

result, _ := pipeline.Run(ctx, "Analyze the market", conv)
```

Summary

```
fmt.Println(tracer.Summary())
```

Output:

```
✓ [manager] pipeline (2.3s) ok
  ✓ [agent] researcher (1.1s) ok
    ✓ [memory] recall (2ms) ok
    ✓ [custom] rag:retrieve (5ms) ok
    ✓ [llm] chat (800ms) ok
    ✓ [tool] web_search (300ms) ok
  ✓ [agent] writer (1.2s) ok
    ✓ [llm] chat (1.1s) ok
```

Stats

```
stats := tracer.Stats()
fmt.Printf("Total spans: %d\n", stats.TotalSpans)
fmt.Printf("LLM calls: %d\n", stats.LLMCalls)
fmt.Printf("Tool calls: %d\n", stats.ToolCalls)
```

Export

```
data := tracer.Export()  // JSON string for analysis/dashboards
```

Span Kinds

Kind	What It Traces
`SpanKindAgent`	Agent execution
`SpanKindTool`	Tool call
`SpanKindLLM`	LLM API call
`SpanKindManager`	Manager orchestration
`SpanKindMCP`	MCP protocol operation
`SpanKindMemory`	Memory recall/store
`SpanKindSession`	Session operations
`SpanKindCustom`	RAG, guardrails, HITL, or your own

Sessions

Save and resume agent conversations across restarts.

Setup

```
store, _ := agentflow.NewFileSessionStore("./sessions")
```

Create and Save

```
session := agentflow.NewSession("market-analysis")
conv := session.GetConversation()
result, _ := agent.Run(ctx, "Analyze AI market", conv)
session.AddResult(result)
session.SetCheckpoint("analyst", map[string]any{"step": 3, "data": "..."})
store.Save(session)
```

Resume Later

```
loaded, _ := store.Load(session.ID)
loaded.Resume()
conv = loaded.GetConversation()  // full history is restored
result, _ = agent.Run(ctx, "Now analyze competitors", conv)
```

Lifecycle States

State	Meaning
`active`	Currently running
`paused`	Saved for later
`completed`	Finished successfully
`failed`	Terminated with error

Session Stores

```
// File-based (JSON files on disk)
store, _ := agentflow.NewFileSessionStore("./sessions")

// In-memory (for testing)
store := agentflow.NewInMemorySessionStore()
```

Eval / Testing Harness

Built-in evaluation framework for testing agent quality and tracking regressions.

Setup

```
suite := agentflow.NewEvalSuite("quality-tests", agent)

suite.AddCase(agentflow.EvalCase{
    Name:  "Answers about Go",
    Input: "What is Go good for?",
    Tags:  []string{"basic"},
    Assertions: []agentflow.Assertion{
        agentflow.AssertContains("concurrent"),
        agentflow.AssertMinLength(100),
        agentflow.AssertMaxDuration(10 * time.Second),
        agentflow.AssertMaxToolCalls(3),
    },
})

report := suite.Run(ctx)
fmt.Println(report.Summary())
```

Output:

```
╔═══════════════════════════════════════════════════╗
║  EVAL REPORT: quality-tests                       ║
║  Total: 5   Passed: 4   Failed: 1                 ║
║  Pass Rate: 80.0%    Duration: 3.2s                ║
╚═══════════════════════════════════════════════════╝
```

Built-in Assertions

Assertion	What It Checks
`AssertContains(text)`	Output contains the text
`AssertNotContains(text)`	Output does not contain the text
`AssertMinLength(n)`	Output is at least n characters
`AssertMaxLength(n)`	Output is at most n characters
`AssertJSON(fields...)`	Output is valid JSON with specific fields
`AssertToolUsed(name)`	A specific tool was called
`AssertToolNotUsed(name)`	A specific tool was not called
`AssertMaxToolCalls(n)`	No more than n tool calls were made
`AssertMaxDuration(d)`	Completed within duration d
`AssertCustom(name, fn)`	Your own assertion function

Regression Tracking

```
// Save baseline
report.ExportJSON("results/v1.json")

// Later: compare against baseline
baseline, _ := agentflow.LoadReport("results/v1.json")
current := suite.Run(ctx)
fmt.Println(agentflow.CompareReports(baseline, current))
```

Run by Tag

```
// Only run cases tagged "critical"
report := suite.RunByTag(ctx, "critical")
```

MCP Integration

MCP (Model Context Protocol) lets agents connect to external tool servers. Tools are auto-discovered — you don't define them manually.

Connect an Agent to MCP

```
agent := agentflow.NewAgent(agentflow.AgentConfig{
    ID:   "researcher",
    Role: "Researcher",
    LLM:  llm,
    MCPServers: []agentflow.MCPServerConfig{
        {
            Name:    "web-search",
            Command: "npx",
            Args:    []string{"-y", "@anthropic/mcp-server-web-search"},
            Env:     map[string]string{"API_KEY": os.Getenv("SEARCH_KEY")},
        },
    },
})
defer agent.Close()
```

Multiple MCP Servers

```
MCPServers: []agentflow.MCPServerConfig{
    {Name: "filesystem", Command: "npx", Args: []string{"-y", "@modelcontextprotocol/server-
    {Name: "database",   Command: "db-mcp-server"},
    {Name: "web-search", Command: "search-server"},
},
```

Manager-Level MCP (Shared)

```
mgr := agentflow.NewManager(agentflow.ManagerConfig{
    Agents: []agentflow.Runnable{agent1, agent2, agent3},
    MCPServers: []agentflow.MCPServerConfig{
        {Name: "shared-search", Command: "search-server"},
    },
})
// All agents get "shared-search" tools automatically
```

Opt Out of MCP

```
agent := agentflow.NewAgent(agentflow.AgentConfig{
    DisableMCP: true,
})
```

Testing with Mock MCP

```
mock := agentflow.NewMockMCPTransport(
    agentflow.MCPToolDef{
        Name: "mcp_search", Description: "Search",
        InputSchema: map[string]any{"type": "object"},
        Handler: func(args map[string]any) string { return "result" },
    },
)
agent.ConnectMCPWithTransport(ctx, "test-server", mock)
```

Conversation & Context

Creating a Conversation

```
conv := agentflow.NewConversation()
result, _ := mgr.Run(ctx, "input", conv)

// Inspect the transcript
for _, turn := range conv.GetTurns() {
    fmt.Printf("[%s] %s: %s\n", turn.AgentID, turn.Role, turn.Content[:80])
}
```

Thread Safety

All Conversation methods are safe for concurrent use — critical for `StrategyParallel` where multiple agents write simultaneously.

Nil Conversation

Passing `nil` is allowed — the agent creates an internal conversation. Use when you don't need the transcript.

```
result, _ := agent.Run(ctx, "Quick question", nil)
```

Error Handling & Reliability

No Hanging — Guaranteed

Every operation has a `context.Context` with a deadline:

```
// Agent level: 2-minute timeout (default)
agent := agentflow.NewAgent(agentflow.AgentConfig{Timeout: 2 * time.Minute})

// Manager level: 10-minute timeout (default)
mgr := agentflow.NewManager(agentflow.ManagerConfig{Timeout: 10 * time.Minute})

// Call level: custom timeout
ctx, cancel := context.WithTimeout(ctx, 30*time.Second)
defer cancel()
result, err := agent.Run(ctx, "query", nil)
```

Max Tool Calls

Prevents runaway tool loops:

```
agent := agentflow.NewAgent(agentflow.AgentConfig{MaxToolCalls: 5})
```

Max Turns (Manager)

Prevents infinite loops in RoundRobin/LLMRouter:

```
mgr := agentflow.NewManager(agentflow.ManagerConfig{MaxTurns: 10})
```

Context Cancellation

```
ctx, cancel := context.WithCancel(ctx)
go func() {
    time.Sleep(10 * time.Second)
    cancel()  // stops all agents immediately
}()
result, err := mgr.Run(ctx, "long task", nil)
```

Logging

AgentFlow uses Go's structured logging (`log/slog`).

```
logger := slog.New(slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
    Level: slog.LevelDebug,
}))

agent := agentflow.NewAgent(agentflow.AgentConfig{Logger: logger})
```

Event	Level	Fields
Agent started	INFO	agent_id, input_length, tool_count
Tool executed	INFO	agent_id, tool, duration, output_length
Agent completed	INFO	agent_id, duration, tool_calls, response_length
Manager started	INFO	manager_id, strategy, agent_count
MCP connected	INFO	mcp_server, tools_discovered

Testing with Mocks

MockLLM

```
func TestMyAgent(t *testing.T) {
    mock := &agentflow.MockLLM{}
    mock.PushResponse(agentflow.LLMResponse{Content: "Market size: $10B"})

    agent := agentflow.NewAgent(agentflow.AgentConfig{
        ID: "test", Role: "Analyst", Goal: "Analyze", LLM: mock,
    })

    result, err := agent.Run(context.Background(), "Analyze AI market", nil)
    if err != nil {
        t.Fatal(err)
    }
    if !strings.Contains(result.Content, "$10B") {
        t.Error("expected market size in result")
    }
}
```

MockMCPTransport

```
mock := agentflow.NewMockMCPTransport(
    agentflow.MCPToolDef{
        Name: "search", Description: "Search the web",
        InputSchema: map[string]any{"type": "object"},
        Handler: func(args map[string]any) string { return "results" },
    },
)
agent.ConnectMCPWithTransport(ctx, "test", mock)
```

Running Tests

```
go test ./...           # all 156 tests
go test -v ./...        # verbose
go test -race ./...     # with race detector
go test -run TestRAG    # specific test
```

Examples Reference

Every example is a standalone Go program in `examples/`. Each has its own `go.mod`.

Example	What It Shows	API Key Needed?	How to Run
`basic`	One agent, one question	No (`--demo`)	`cd examples/basic && go run main.go --demo`
`single_agent`	Agent with a custom tool	No (MockLLM)	`cd examples/single_agent && go run main.go`
`two_agent`	Sequential pipeline: researcher → writer	No (MockLLM)	`cd examples/two_agent && go run main.go`
`parallel_agents`	6 strategy agents in parallel	No (MockLLM)	`cd examples/parallel_agents && go run main.go`
`llm_router`	LLM dynamically picks agents	No (MockLLM)	`cd examples/llm_router && go run main.go`
`external_api`	Agent calling HTTP APIs as tools	No (MockLLM)	`cd examples/external_api && go run main.go`
`mcp_agent`	MCP: 4 integration patterns	No (MockMCP)	`cd examples/mcp_agent && go run main.go`
`rag`	RAG: files, URLs, dynamic uploads	No (`--demo`)	`cd examples/rag && go run main.go --demo`
`showcase`	Multi-agent + MCP web search	Yes (Mistral)	`cd examples/showcase && go run main.go`
`frameworks`	7 strategy frameworks + synthesizer	Yes (Mistral)	`cd examples/frameworks && go run main.go`
`webui`	WhatsApp-style chat with HITL	Yes (Mistral)	`cd examples/webui && go run main.go`

API Reference

Core Types

```
AgentConfig{ID, Role, Goal, Backstory, Tools, LLM, MaxToolCalls, Timeout,
            Logger, MCPServers, DisableMCP, Memory, HITL,
            InputGuardrails, OutputGuardrails, RAG}

NewAgent(config) → *Agent
agent.Run(ctx, input, conv) → (*Result, error)
agent.GetID() → string
agent.Close() → error

ManagerConfig{ID, Strategy, Agents, MaxTurns, Timeout, LLM, Logger, MCPServers}
NewManager(config) → *Manager
manager.Run(ctx, input, conv) → (*Result, error)
manager.AddAgent(agent)
manager.Close() → error

Result{AgentID, Content, Metadata}
```

Strategies

```
StrategySequential    — one after another
StrategyParallel      — all at once
StrategyRoundRobin    — take turns
StrategyLLMRouter     — LLM decides
```

RAG

```
NewInMemoryVectorStore() → *InMemoryVectorStore
NewTextLoader(path) → *TextLoader
NewDirectoryLoader(dir, exts) → *DirectoryLoader
NewStringsLoader(map) → *StringsLoader
NewURLLoader(url) → *URLLoader
NewMultiURLLoader(urls...) → *multiURLLoader
NewIngestionPipeline(config) → *IngestionPipeline
IngestDocuments(ctx, store, chunker, embedder, docs...) → (stats, error)
SimpleRAG(store, topK) → *RAGConfig
DefaultChunker() → Chunker

NewFixedSizeChunker(size, overlap) → *FixedSizeChunker
NewParagraphChunker() → *ParagraphChunker
NewMarkdownChunker() → *MarkdownChunker
NewRecursiveChunker(size, overlap) → *RecursiveChunker

NewNoOpEmbedder() → *NoOpEmbedder
NewOpenAIEmbedder(config) → *OpenAIEmbedder
NewMistralEmbedder(config) → *MistralEmbedder

NewKeywordRetriever(store) → *KeywordRetriever
NewVectorRetriever(store) → *VectorRetriever
NewHybridRetriever(store, alpha) → *HybridRetriever
```

Memory

```
NewInMemoryStore() → *InMemoryStore
MemoryEntry{Kind, Content, Tags, TTL, SessionID, AgentID, Priority}
MemoryFact, MemoryEpisode, MemoryInstruction, MemorySummary
store.Store(ctx, entry) → error
store.Search(ctx, query, limit) → ([]MemoryEntry, error)
store.Export() → ([]byte, error)
store.Import(data) → error
```

Guardrails

```
NewGuardrailChain(guardrails...) → *GuardrailChain
chain.Run(ctx, input) → (string, error)

MaxLength(n), MinLength(n), BlockPatterns(regex...),
RequirePatterns(regex...), BlockKeywords(words...),
RequireJSON(fields...), ContentFilter(types...),
RedactPII(), TrimWhitespace(), CustomGuardrail(name, fn)
```

HITL

```
HITLConfig{Policy, DangerousTools, Handler, CustomPolicy}
NewHITLController(config) → *HITLController
NewCLIApprovalHandler() → *CLIApprovalHandler
NewChannelHITLHandler() → *ChannelHITLHandler
AutoApproveHandler{}

HITLApprove, HITLReject, HITLEdit, HITLApproveAll, HITLAbort
HITLPolicyNone, HITLPolicyAllTools, HITLPolicyDangerous, HITLPolicyCustom
```

Tracing

```
NewTracer() → *Tracer
ContextWithTracer(ctx, tracer) → context.Context
StartSpanFromContext(ctx, name, kind) → (*Span, context.Context)
tracer.Summary() → string
tracer.Stats() → TraceStats
tracer.Export() → string
tracer.Spans() → []*Span

SpanKindAgent, SpanKindTool, SpanKindLLM, SpanKindManager,
SpanKindMCP, SpanKindMemory, SpanKindSession, SpanKindCustom
```

Sessions

```
NewSession(name) → *Session
session.GetConversation() → *Conversation
session.AddResult(result)
session.SetCheckpoint(key, data)
session.GetCheckpoint(key) → (data, bool)
session.Resume(), session.Pause(), session.Complete(), session.Fail(err)

NewFileSessionStore(dir) → (*FileSessionStore, error)
NewInMemorySessionStore() → *InMemorySessionStore
store.Save(session) → error
store.Load(id) → (*Session, error)
store.List() → ([]string, error)
store.Delete(id) → error
```

Eval

```
NewEvalSuite(name, agent) → *EvalSuite
suite.AddCase(case)
suite.Run(ctx) → *EvalReport
suite.RunByTag(ctx, tag) → *EvalReport
report.Summary() → string
report.ExportJSON(path) → error
LoadReport(path) → (*EvalReport, error)
CompareReports(baseline, current) → string

AssertContains(text), AssertNotContains(text),
AssertMinLength(n), AssertMaxLength(n), AssertJSON(fields...),
AssertToolUsed(name), AssertToolNotUsed(name),
AssertMaxToolCalls(n), AssertMaxDuration(d), AssertCustom(name, fn)
```

FAQ

Q: Does AgentFlow require a specific LLM provider? No. OpenAI and Mistral are built in. Use `NewOpenAI` with a custom `BaseURL` for Groq, Together AI, Ollama, or any OpenAI-compatible API. Implement the `LLM` interface for non-compatible providers.

Q: Can I run it without an API key? Yes. Use `MockLLM` for testing. 8 of 11 examples work without any API key.

Q: Will my agents hang? No. Every operation has a `context.Context` deadline. If anything exceeds its deadline, it returns an error.

Q: Can different agents use different LLMs? Yes. Each agent has its own `LLM` field.

Q: Does RAG require a vector database? No. `InMemoryVectorStore` works out of the box with keyword search. Embeddings are optional.

Q: Can I add documents to RAG mid-conversation? Yes. Call `agentflow.IngestDocuments()` at any time — the agent's next query will search the new documents.

Q: Can I load documents from a URL? Yes. Use `NewURLLoader("https://...")` or `NewMultiURLLoader(urls...)`. HTML is automatically stripped.

Q: Is it thread-safe? Yes. Conversation, MCP, and vector store all use proper synchronization. All 156 tests pass with Go's race detector.

Q: What reasoning pattern do agents use? ReAct (Reason → Act → Observe). The agent calls the LLM, optionally executes tools, feeds results back, and loops until the LLM returns a final answer.

Q: Can I nest managers? Yes, to any depth. Managers implement `Runnable`, so a manager's agent list can include other managers.

Comparison

AgentFlow vs Other Frameworks

Feature	AgentFlow (Go)	CrewAI (Python)	AutoGen (Python)	Eino (Go)	Google ADK (Go)
Language	**Go**	Python	Python	Go	Go
Dependencies	**Zero**	50+	30+	CloudWeGo	Google Cloud
Multi-agent	**4 strategies**	Yes	Yes	Chain/Graph	Sub-agents
RAG	**Built-in**	Plugin	Plugin	External	Vertex
Memory	**Multi-layer**	Basic	Basic	Redis	Vertex RAG
Tracing	**OTel-style**	No	No	Callbacks	OpenTelemetry
HITL	**Approve/Reject/Edit**	No	Basic	Interrupt	Confirmation
Guardrails	**10 built-in**	No	No	Composition	Plugins
Eval	**Built-in**	No	No	External	Dev UI
Sessions	**Save/Resume**	No	No	Context	Events
MCP	**Built-in**	Plugin	Plugin	Extension	Toolbox
Per-agent LLM	**Yes**	No	No	Yes	Yes
Binary size	**~8 MB**	~200 MB+	~200 MB+	Varies	Varies
Tests	**156**	Varies	Varies	Varies	Varies

AgentFlow — Build AI agent systems in Go that don't hang.